UNIVERSIDAD DE CALDAS — Programación Web Creativa II — TALLER DE EVALUACIÓN			
Asignatura	Programación Web Creativa II	**Docente**	Andrés Felipe Osorio Quintero
Perfil docente	Ingeniero de Software, desarrollador web y marketing digital	**Unidad**	Unidad 1: Manipulación del DOM y manejo de eventos
Subtema	Gestión de eventos en el navegador	**Documento fuente**	3-Gestión de eventos en el navegador.pdf
Código base	Codigo HTML CSS JS.docx	**Estudiantes**	**Valentina Muñoz Estrada & Kevin Carmona**
Fecha	27 de Octubre, 2026		

Responda las preguntas 1 a 5 con sus propias palabras. Desarrolle los ejercicios 6 a 20 en Visual Studio Code y adjunte las evidencias solicitadas debajo de cada uno.

PARTE I · TEORÍA (PREGUNTAS 1 A 5)

1. Programación dirigida por eventos

Una página web pasa de ser un documento a comportarse como una aplicación gracias al modelo de eventos. Explique con sus propias palabras en qué consiste la programación dirigida por eventos y cómo intervienen el hilo principal y el bucle de eventos (event loop). Argumente por qué escribir manejadores breves es una cuestión de experiencia de usuario y no solo de elegancia del código.

Respuesta	Es un modelo donde el flujo del programa lo determinan las acciones del usuario (clics, teclas). El **hilo principal** ejecuta el JS síncrono y renderiza la interfaz. El **event loop** monitorea la pila y traslada funciones de la cola de tareas cuando la pila se vacía. Manejadores cortos previenen el bloqueo del hilo principal, evitando congelar la interfaz (lag) y garantizando fluidez.

2. Formas de registrar manejadores

El documento presenta tres formas de asociar código a un evento: el atributo HTML, la propiedad del objeto y addEventListener. Compárelas e indique por qué addEventListener es la forma recomendada en proyectos actuales. Incluya el papel de las opciones capture, once, passive y signal, y explique qué condiciones deben cumplirse para retirar un manejador con removeEventListener.

Respuesta	Los *atributos HTML* mezclan capas; las *propiedades DOM* permiten solo un manejador. `addEventListener` es el estándar porque admite múltiples escuchadores y opciones: `capture` (fase previa), `once` (autoremoción), `passive` (mejora scroll) y `signal` (cancelación con AbortController). Para usar `removeEventListener` se debe enviar la **misma referencia de función** y fase.

3. Propagación y control del flujo

Todo evento recorre el árbol del DOM en fases. Explique las fases de captura, objetivo y burbujeo, y la diferencia entre target y currentTarget. Luego distinga preventDefault() de stopPropagation() y argumente por qué conviene evitar esta última, indicando cómo puede resolverse el problema sin detener la propagación.

Respuesta	Fases: **Captura** (desciende desde la raíz), **Objetivo** (llega al nodo), **Burbujeo** (asciende). `target` es el origen real del evento y `currentTarget` el nodo del listener. `preventDefault()` frena la acción por defecto; `stopPropagation()` interrumpe el viaje por el DOM. Se aconseja evitarla porque rompe la delegación y analíticas globales. Se resuelve validando condicionalmente con `event.target`.

4. Delegación de eventos

Una lista de tareas crea elementos nuevos mientras la aplicación está en uso. Explique qué es la delegación de eventos, qué problema resuelve frente al registro de un manejador por elemento y cómo se aplica el procedimiento (ancestro, closest, contains y atributos data-*). Analice en qué situaciones no es la mejor opción.

Respuesta	Consiste en poner un único listener en un ancestro común aprovechando el burbujeo. Evita consumo excesivo de memoria y fugas en elementos dinámicos. Se aplica usando `e.target.closest()` para ubicar el hijo objetivo y `dataset` para leer datos. No conviene en eventos sin burbujeo (`focus`, `mouseenter`) o con cálculos de selecciones muy complejos en scroll.

5. Validación, accesibilidad y calidad de los datos

Explique la diferencia entre la validación en el cliente y en el servidor, y por qué la validación del navegador nunca debe considerarse un control de seguridad. Describa cómo la Constraint Validation API y los atributos ARIA contribuyen a mostrar mensajes de error accesibles, y relacione estos aspectos con la calidad de los datos de origen en un proceso ETL.

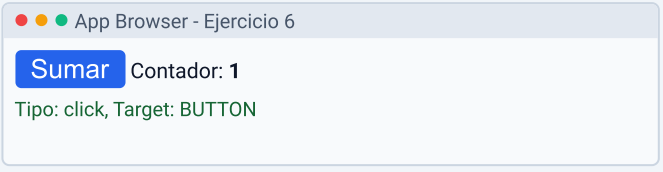
Respuesta	La validación en cliente mejora UX; la de servidor garantiza seguridad absoluta ya que la del cliente se puede deshabilitar. La **Constraint Validation API** y atributos ARIA (`aria-invalid`, `aria-describedby`) anuncian errores a lectores de pantalla. En procesos ETL, validar datos en el origen asegura registros limpios y previene errores en las etapas de transformación e ingesta.

PARTE II · PRÁCTICA (PREGUNTAS 6 A 20)

6. Registro de manejadores y objeto de evento

Actividad: Cree un botón «Sumar» y un contador visible. Registre el manejador con `addEventListener` usando una función con nombre. Al pulsar, el contador aumenta y un mensaje (`role="status"`) indica el `type` y el `target`.

```
// Ex6: Listener nombrado let count = 0; function sumar(e) { count++; counterEl.textContent = count; statusEl.textContent = `Tipo: ${e.type}, Target: ${e.target.tagName}`; } btnSumar.addEventListener( 'click', sumar );
```

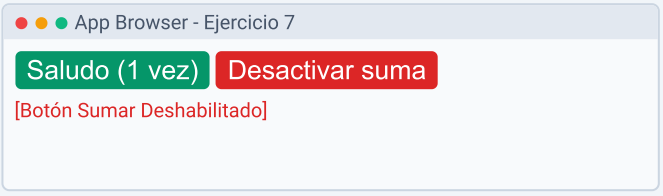


Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

7. Manejadores de un solo uso y retirada de manejadores

Actividad: Agregue un botón «Saludo» (opción `once`) y un botón «Desactivar suma» que retire con `removeEventListener` el manejador del ejercicio 6 y deshabilite el botón «Sumar».

```
// Ex7: Option once y removeEventListener btnSaludo.addEventListener( 'click', () => alert('¡Hola!'), { once: true } ); btnDesactivar.addEventListener('click', () => { btnSumar.removeEventListener('click', sumar); btnSumar.disabled = true; });
```

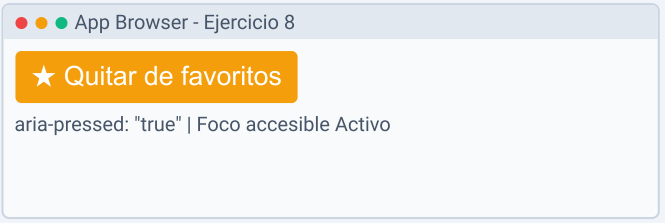


Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

8. Botón de dos estados accesible

Actividad: Use un `button` nativo que alterne su texto («Agregar a favoritos» / «Quitar de favoritos») y su atributo `aria-pressed` en cada clic. Operable con Tab, Enter y Espacio.

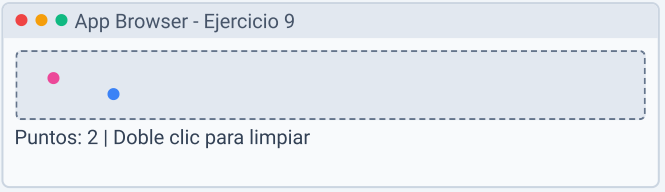
```
// Ex8: Botón accesible ARIA btnFav.addEventListener('click', () => { const isPressed = btnFav.getAttribute('aria-pressed') === 'true'; btnFav.setAttribute('aria-pressed', !isPressed); btnFav.textContent = !isPressed ? 'Quitar de favoritos' : 'Agregar a favoritos'; });
```



9. Coordenadas del clic y doble clic

Actividad: Lienzo rectangular que dibuja un punto de color en la posición del clic con `clientX/Y` y `getBoundingClientRect()`. Muestra total de puntos y limpia con `dblclick` usando `replaceChildren()`.

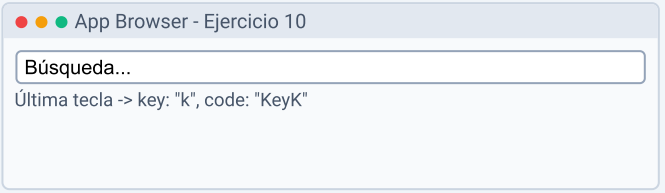
```
// Ex9: Coordenadas del lienzo canvas.addEventListener('click', (e) => { const rect = canvas.getBoundingClientRect(); const x = e.clientX - rect.left; const y = e.clientY - rect.top; drawDot(x, y); }); canvas.addEventListener('dblclick', () => { canvas.replaceChildren(); });
```



10. Atajo de teclado y diferencia entre key y code

Actividad: Campo de búsqueda enfocado con Ctrl+K (o Cmd+K) llamando a `preventDefault()`. Borra con Escape. Muestra `key` y `code` e ignora teclas en composición (`isComposing`).

```
// Ex10: Atajo Ctrl+K & Escape document.addEventListener('keydown', (e) => { if (e.isComposing) return; if ((e.ctrlKey || e.metaKey) && e.code === 'KeyK') { e.preventDefault(); searchInput.focus(); } if (e.key === 'Escape') { searchInput.value = ''; searchInput.blur(); } });
```

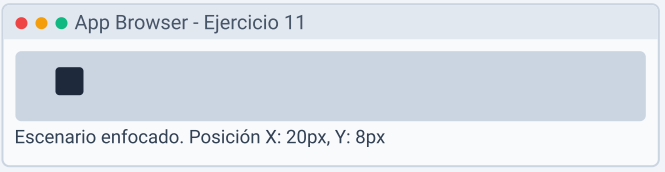


Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

11. Movimiento con las flechas del teclado

Actividad: Escenario enfocable (`tabindex="0"`) con un cuadro que se mueve con las flechas sin salirse. Llama a `preventDefault()` solo para las flechas gestionadas.

```
// Ex11: Flechas de movimiento stage.addEventListener('keydown', (e) => { const keys = [ 'ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight' ]; if (keys.includes(e.key)) { e.preventDefault(); moveBox(e.key); } });
```



Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

12. Barra de progreso de lectura con scroll

Actividad: Barra de progreso fija con `requestAnimationFrame`. Botón «Volver arriba» visible desde 400px que respeta `prefers-reduced-motion`.

```
// Ex12: Progressbar con rAF let ticking = false;
window.addEventListener('scroll', () => { if (!ticking) {
window.requestAnimationFrame(() => { updateProgress(); tic
king = false; }); ticking = true; } });
```

App Browser - Ejercicio 12

Progreso Scroll: 65% | [↑ Volver arriba visible]

13. Revelado de secciones con IntersectionObserver

Actividad: Revelado de secciones con `IntersectionObserver` (threshold 0.15) e `unobserve`. Degradación progresiva agregando clases desde JS.

```
// Ex13: IntersectionObserver const observer = new
IntersectionObserver( (entries) => { entries.forEach(entry
=> { if (entry.isIntersecting) { entry.target.classList.ad
d('visible'); observer.unobserve(entry.target); } }); }, {
threshold: 0.15 } );
```

App Browser - Ejercicio 13

✓ Sección 1 Revelada (unobserved)

14. Eventos de formulario y lectura con FormData

Actividad: Formulario de contacto con contador de caracteres restantes (`input`), resalte activo (`focusin/out`) y captura de datos con `FormData` a JSON.

```
// Ex14: Lectura FormData JSON form.addEventListen
er('submit', (e) => { e.preventDefault(); const data = new
FormData(form); const json = Object.fromEntries(data); con
sole.log(JSON.stringify(json)); form.reset(); });
```

App Browser - Ejercicio 14

```
{"nombre":"Valentina","mensaje":"Hola..."}
```

Caracteres restantes: 185

Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

15. Captura, burbujeo y stopPropagation

Actividad: Cajas anidadas (A, B, C) para inspeccionar la fase (`eventPhase`) y probar detención de propagación con `stopPropagation()` en B.

```
// Ex15: Detener propagación boxB.addEventListener
( 'click', (e) => { logPhase('Box B', e.eventPhase); if (s
topCheck.checked) { e.stopPropagation(); } }, useCapture
);
```

App Browser - Ejercicio 15

Box A -> Box B [stopPropagation] -X (Box C no alcanzado)

Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

16. Lista de tareas con delegación de eventos

Actividad: Delegación de eventos en un `ul` único para botones eliminar y casillas con `closest()` y `contains()`.

```
// Ex16: Delegación en UL todoList.addEventListener('click', (e) => { const btn = e.target.closest('.btn-delete'); if (btn && todoList.contains(btn)) { btn.closest('li').remove(); } });
```

App Browser - Ejercicio 16

- [x] ~~Tarea 1~~ [X]

Completadas: 1/1

17. Menú de navegación interactivo

Actividad: Menú responsivo con patrón de divulgación, `aria-expanded`, cierre con Escape y clic externo sin usar `stopPropagation()`.

```
// Ex17: Clic fuera del menú document.addEventListener('click', (e) => { if (!nav.contains(e.target)) { menu.setAttribute('hidden', ''); btnMenu.setAttribute( 'aria-expanded', 'false' ); } });
```

App Browser - Ejercicio 17

≡ Menú (aria-expanded="false")

Oculto en móvil / Visible en >700px

18. Formulario de registro con validación accesible

Actividad: Validación accesible con `novalidate`, `setCustomValidity` en contraseñas, `aria-invalid` y `aria-describedby`.

```
// Ex18: Custom Validity ARIA passConfirm.addEventListener('input', () => { const match = passConfirm.value === pass.value; passConfirm.setCustomValidity( match ? '' : 'No coinciden' ); passConfirm.setAttribute( 'aria-invalid', !match ); });
```

App Browser - Ejercicio 18

•••

⚠ Las contraseñas no coinciden

Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

19. Galería dinámica con filtros y visor modal

Actividad: Renderizado seguro con `DocumentFragment` y modal nativo `dialog` con `showModal()`.

```
// Ex19: Fragment & Modal const frag = document.createDocumentFragment(); items.forEach(item => { const el = document.createElement('div'); el.textContent = item.title; frag.appendChild(el); }); gallery.replaceChildren(frag);
```

App Browser - Ejercicio 19

Visor Modal <dialog> Activo

Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

20. Proyecto integrador: panel administrativo de usuarios

Actividad: Panel de usuarios con detección de duplicados, filtro en vivo con atajo "/", y exportación a CSV escapando caracteres de fórmula.

```
// Ex20: Escaping CSV function exportCSV(users) {
const rows = users.map(u => { const safeName = /^[=+@-]/.test(u.name) ? `'${u.name}` : u.name; return `"${safeName}","${u.email}"`; }); }
```

App Browser - Ejercicio 20

Nombre	Email
Valentina M.	valen@ucaldas.edu.co
Kevin C.	kevin@ucaldas.edu.co

Exportar CSV

Enlace de GitHub (opcional): https://github.com/valentina-kevin/web-events-workshop

ENTREGA FINAL

Pegue el enlace completo del repositorio de GitHub donde se encuentran los ejercicios 6 a 20:

Enlace del repositorio de GitHub:	